全国大学生嵌入式芯片与系统设计竞赛

AMD 赛道 · 作品设计报告

基于 FPGA 视觉—惯性协同的自主避障机器人

PYNQ-Z2(Zynq-7020)· PL 实时感知与控制 · YOLO 目标检测

项目	内容
参赛队名	【待补:队伍名称】
参赛学校	【待补:学校与学院】
队员	【待补:队员姓名】
指导教师	【待补:指导教师】
赛道 / 题目	AMD 赛道 · 自主选题
提交日期	2026 年 10 月

摘 要

面向移动机器人在室内场景中的实时避障需求,本作品以 Xilinx Zynq-7020(PYNQ-Z2)异构平台为核心,设计并实现了一台基于 FPGA 视觉—惯性协同的自主避障小车。系统把“必须确定、必须及时”的工作放进可编程逻辑(PL),把“需要算力”的目标检测放到外部 PC(预留迁移至 AMD Ryzen AI PC),把策略与调度留给 ARM(PS),形成 PL、PS、PC 三层分工。

在 PL 中,我们自主设计了两个 Verilog 核并完成板级验证:(1)IMU 读取核,以 I²C 400 kHz 每 2 ms 采集一次惯性数据,完成上电零偏校准与偏航角积分,并为每一帧提供微秒级硬件时间戳;(2)电机控制核,实现双路 PWM、死区补偿、方向与刹车控制,以及 200 ms 无心跳自动刹车的硬件看门狗。此外,使用 Vitis HLS 实现了 640×480→320×240 的图像预处理核,并通过 AXI DMA 完成了与 ARM 软件实现的对比评测。

实测结果表明:PL 时间戳在 1497 个相邻样本中间隔全部为 2000 μs(标准差 0);PS 向 PL 写一次电机命令的 P50 为 23.74 μs;从“画面到手”到“电机指令写出”的端到端延迟 P50/P95/P99/max 分别为 14.13 / 18.93 / 23.48 / 27.32 ms;闭环转向误差中位数 1.77°;HLS 预处理核相对 ARM 软件基线取得约 4.0 倍(核心)和 1.88 倍(含 DMA 端到端)的加速。整个设计仅占用 8.32% LUT、6.33% 寄存器、2.14% BRAM、2.27% DSP,时序裕量 WNS=+1.024 ns,Vivado 估算总功耗 1.432 W(其中 PL 用户逻辑约 0.038 W)。

本报告同时如实说明了系统的边界:目标检测依赖 COCO 预训练类别;预处理核当前作为基准评测而未接入主控制环;完整的避障成功率统计待补充(见附录 D)。

关键词: Zynq-7020;FPGA;视觉—惯性协同;自主避障;硬件时间戳;硬件看门狗;HLS;YOLO

目 录

(提示:用 Word 打开后如目录为空,请单击目录区域后按 F9,选择“更新整个目录”即可。)

1 项目背景与意义

1.1 研究背景

自主移动机器人要在真实环境中可靠运行,首先要解决“看见障碍并及时避开”的问题。视觉是获取环境语义最丰富的手段,但视觉处理的延迟往往不稳定;惯性测量单元(IMU)能以很高的频率给出机体姿态变化,却会随时间漂移。如何让两类传感器的数据在统一的时间基准下被使用,并让执行机构在计算延迟波动时仍然安全,是嵌入式机器人系统设计的核心难点。

传统方案把所有任务交给一颗通用处理器:在 Linux 上用 Python 或 C++ 轮询传感器计算控制量、驱动电机。这样做有三个固有缺陷:其一,操作系统调度带来不可预期的抖动;其二,一旦上层进程卡死,电机可能保持上一条命令继续运动,存在安全隐患;其三,不同传感器之间缺少硬件级的统一时间戳,后续做视觉—惯性数据对齐只能依赖软件时间,误差难以量化。

1.2 我们的思路

FPGA 的优势恰恰是“确定性”:硬件逻辑的时序由时钟决定,不受操作系统调度影响,且可以并行完成多项任务。因此我们把系统中**对确定性和安全性要求最高的部分**下沉到 PL:传感器的定时采集与打时间戳、电机 PWM 的产生,以及“上层失联就自动刹车”的看门狗。把**需要大算力**的目标检测放到 PC(可迁移至 AMD Ryzen AI PC),把**需要灵活调整**的避障策略留在 ARM 上用 Python 实现。三者各司其职,通过 AXI 总线与以太网连接。

> **一句话概括:** 让 FPGA 做“不能错、不能慢”的事,让 AI PC 做“算得多”的事,让 ARM 做“想得巧”的事。

1.3 作品定位与创新价值

- **可量化的确定性:** 硬件时间戳间隔标准差为 0,PS→PL 控制写延迟稳定在 24 μs 量级,所有指标均给出 P50/P95/P99/max 与原始日志。
- **硬件级安全:** 看门狗在 PL 中实现,独立于 Linux 与 Python,上层卡死或掉线后最长 200 ms 内自动刹车。
- **资源极低:** 全部自研与 IP 逻辑仅占器件 8.3% LUT,PL 用户逻辑功耗约 38 mW,为后续扩展(如把预处理接入主环、加入更多传感器)留足空间。
- **可复现:** 提供完整源码、比特流、日志与傻瓜式操作手册,他人拿到小车即可复现全部实验。

2 需求分析与设计目标

根据竞赛对“FPGA 必须承担真实工作、并给出可复现的量化指标”的要求,我们把需求拆成功能、性能和可靠性三类,并为每一项设定了可检验的目标。

表 2-1 设计目标与实测结果对照

类别	指标	设计目标	实测结果	结论
功能	自主避障	检测到前方目标后转向避开	已实现,水瓶、人等目标均触发转向	达成
功能	航向保持	直行偏航小	直行段偏航角波动约 ±5°(试验日志)	达成
功能	惯性时间戳	每帧带硬件时间戳	PL 微秒时间戳,间隔 2000 μs(σ=0)	达成
性能	端到端延迟	P99 小于一个帧周期(40 ms)	P50 14.13 / P99 23.48 ms	达成
性能	控制写延迟	微秒量级	P50 23.74 μs	达成
性能	转向精度	误差 ⩽ 3°	中位数 1.77°,21/23 次 ⩽3°	基本达成
可靠性	失联保护	200 ms 内自动刹车	PL 看门狗,LD0 指示	达成
资源	PL 占用	低于 20% LUT	8.32% LUT	达成
统计	避障成功率	每类目标 ⩾10 次	**【待补:水瓶成功率 __/__】**	待补

3 系统总体方案

3.1 总体架构

系统由三层构成(图 3-1)。PL 层包含自研的 IMU 读取核、电机控制核,以及 AXI DMA 与 HLS 预处理核;PS 层运行 Linux 与 PYNQ,负责摄像头采集、避障决策与闭环控制;PC 层运行 YOLOv8n 检测服务。PS 与 PL 之间用 AXI-Lite 寄存器通信,PS 与 PC 之间用以太网 TCP 通信。

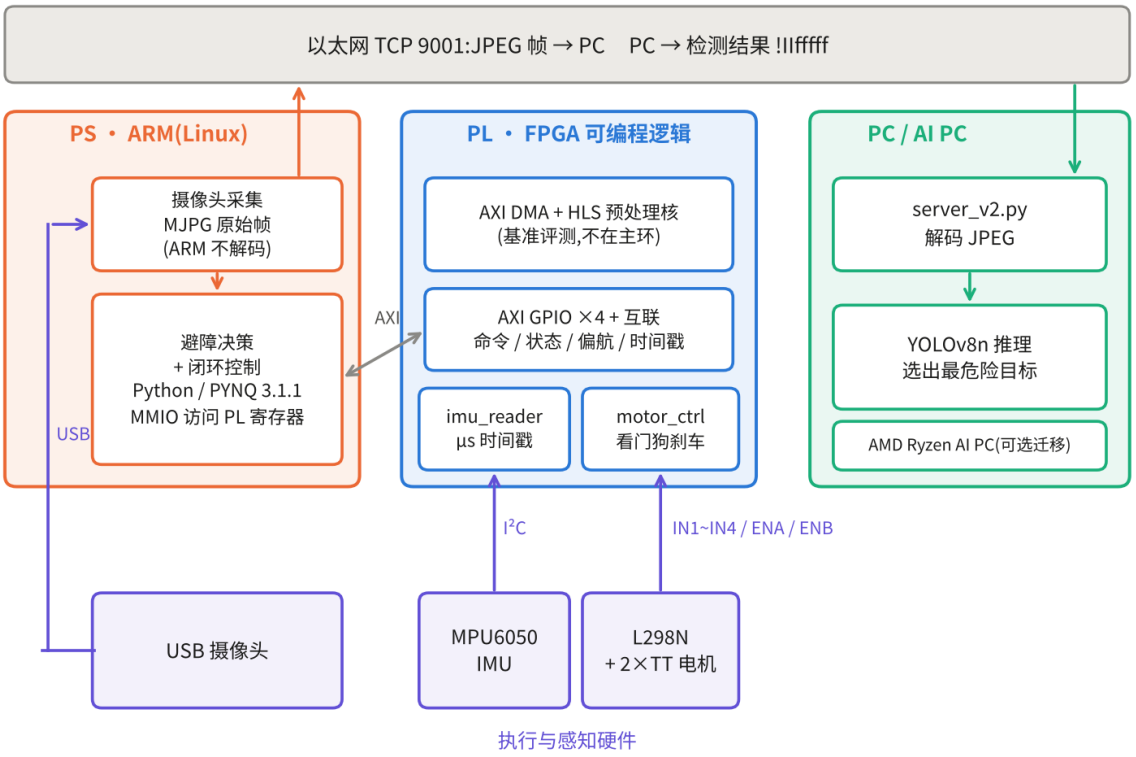


图 3-1　系统总体架构(PS / PL / PC 三层分工)

3.2 为什么这样划分

表 3-1　功能划分及理由

功能	放在哪里	理由
IMU 定时采集、零偏校准、偏航积分、打时间戳	PL	需要严格等间隔,软件定时有抖动;时间戳要由硬件给出才可信
电机 PWM 与方向控制、死区补偿	PL	PWM 本身就是硬件擅长的任务,且不受 Linux 调度影响
失联看门狗	PL	必须独立于可能卡死的软件,才能真正提供安全保障
图像预处理(缩放)	PL(HLS,基准评测)	数据并行度高,适合硬件;因端到端收益有限,暂未接入主环
目标检测(YOLOv8n)	PC(可迁移至 AI PC)	需要较大算力;PYNQ-Z2 的 ARM 不适合实时跑检测网络
避障决策与转向策略	PS(Python)	逻辑易变、需要快速迭代,软件实现最灵活

3.3 数据流与控制流程

一次完整的避障循环如图 3-2:摄像头输出的 MJPG 原始帧由 ARM 直接转发(ARM 不解码,约 17 KB/帧),经 TCP 发给 PC;PC 解码并用 YOLOv8n 推理,选出最危险的目标,返

回目标数、水平位置、面积占比、推理耗时、底边位置与类别;PS 依据面积阈值判定是否转向,并在 PL 的偏航角反馈下闭环执行;PL 电机核输出 PWM,同时检测心跳。全程以 PL 微秒时间戳打点,便于事后复盘。

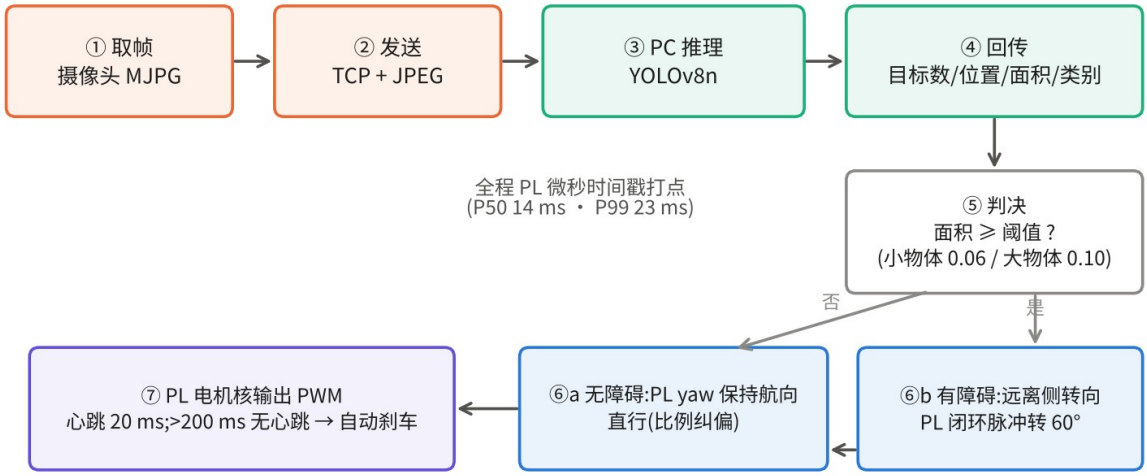


图 3-2 避障控制流程

3.4 软硬件接口

PS 通过 4 组 AXI GPIO 与 PL 通信(表 3-2)。电机命令由 PS 周期性写入,同时翻转一个心跳位;偏航角与时间戳由 PL 持续更新,PS 使用“读序号—读数据—再读序号”的无锁快照方式读取,保证数据一致。

表 3-2 AXI 地址映射

外设	基地址	通道 1	通道 2
axi_gpio_0	0x4120_0000	占空比字(电机 A/B,各 0~1000)	控制字(方向、使能、心跳)
axi_gpio_1	0x4121_0000	配置字(死区、调试选择、校准请求)	硬件时间戳
axi_gpio_2	0x4122_0000	状态字(初始化、WHO_AM_I、校准、序号)	偏航角(毫度,有符号)
axi_gpio_3	0x4123_0000	原始数据寄存器	PL 当前微秒计数
axi_dma_0	0x41E0_0000	MM2S / S2MM(简单模式)	—

4 硬件设计

4.1 平台与器件

表 4-1 主要硬件清单

部件	型号 / 规格	作用
主控平台	PYNQ-Z2(XC7Z020-1CLG400)	PS(双核 ARM Cortex-A9)+ PL(53,200 LUT)
惯性传感器	GY-521 模块(MPU-6050 兼容,WHO_AM_I=0x70)	提供角速度,PL 积分得偏航角
电机驱动	L298N 双路驱动	驱动两个 TT 减速电机
执行机构	2 个 TT 减速电机 + 塑料轮	差速驱动
视觉传感器	USB 摄像头(输出 MJPG 640×480)	采集前方图像
检测主机	PC(YOLOv8n);可迁移至 AMD Ryzen AI PC	运行目标检测
电源	开发板 5 V 适配器;电机独立电池组	供电隔离,减少相互干扰

4.2 引脚与接线

表 4-2 PL 引脚分配

信号	连接	PYNQ-Z2 引脚	说明
IN1 / IN2 / IN3 / IN4	L298N 方向控制	Y17 / Y16 / Y19 / Y18(PMODA)	电机 A、B 方向
ENA / ENB	L298N 使能(PWM)	W19 / W18(PMODA)	PL 输出 PWM
SCL	IMU 时钟	V12(PMODB)	I²C,开漏
SDA	IMU 数据	W13(PMODB)	I²C,开漏,双向
LD0	看门狗指示灯	板载 LED	看门狗触发时点亮

> **经验教训:** IMU 的 SDA/SCL 是双向开漏信号。我们的块设计在综合时把 PL 模块作为独立综合单元(out-of-context),三态信号未能自动映射到引脚,导致读到的引脚电平恒为 0。最终的解决办法是在 RTL 中显式例化 `IOBUF` 原语。这个问题只在上板时才暴露,仿真无法发现。

4.3 供电与可靠性

为避免电机启停造成的电压跌落影响开发板,电机使用独立电池组供电,与开发板仅共地。开发过程中我们发现,摄像头出图瞬间的电流冲击在供电裕量不足时会导致开发板复位;改用更稳定的供电并避免重复下载比特流后,系统在连续多轮试验中保持稳定(见 9.1 节)。

5 FPGA 设计与实现

5.1 IMU 读取核(imu_reader)

该核用有限状态机实现 I²C 主机,以 400 kHz 速率读取 IMU 的加速度、温度与角速度寄存器。采样周期固定为 2000 μs,由 PL 内部计数器严格定时,与软件无关。核的功能包括:上电初始化与 WHO_AM_I 校验;连续采样;上电静止时的角速度零偏估计;偏航角积分(单位毫度,逆时针为正);以及为每个样本附加 PL 微秒时间戳。

为让 ARM 在不阻塞 PL 的前提下读到一致的数据,我们采用 **seqlock(序号锁)快照**:PL 在同一个时钟周期内一次性提交偏航角、时间戳等全部结果,并让序号加 1;ARM 读取时先读序号、再读数据、最后再读序号,两次序号相同,说明期间没有新样本提交,数据来自同一个样本。这样既不需要握手,也不会读到“半新半旧”的数据。

表 5-1 状态字(axi_gpio_2 通道 1)

位	含义
[0]	init_done:初始化完成
[1]	who_ok:WHO_AM_I 校验通过(0x70)
[2]	calibrating:零偏校准中
[3]	calibrated:校准完成
[4]	last_poll_nack:最近一次轮询收到 NACK
[15:8]	miss:丢失的采样周期数
[31:16]	seq:采样序号(seqlock)

5.2 电机控制核(motor_ctrl)

电机控制核接收 PS 写入的 16 位占空比(0~1000)与控制字,输出两路 PWM 与四路方向信号。方向支持滑行、正转、反转、刹车四种状态。**死区补偿**:TT 电机在低占空比下不转,核内将非零命令抬升到可配置的最小占空比,使小命令也有响应。

硬件看门狗是该核最重要的安全特性:PS 每 20 ms 翻转一次控制字的心跳位;如果 PL 检测到心跳超过 200 ms 没有翻转(上层进程崩溃、网络阻塞或系统卡死),就自动把两路电机置为刹车,并点亮 LD0。由于这段逻辑完全在 PL 内,不依赖 Linux 是否存活,因此是真正意义上的安全兜底。

表 5-2 控制字(axi_gpio_0 通道 2)

位	含义
[1:0]	电机 A 方向:00 滑行 / 01 正转 / 10 反转 / 11 刹车
[3:2]	电机 B 方向(同上)

位	含义
[4]	使能
[5]	心跳位(需周期翻转,否则触发看门狗)

5.3 HLS 图像预处理核与 DMA

我们用 Vitis HLS 实现了 `preproc` 核:AXI-Stream 接口,32 位宽,输入 640×480 BGR888,输出 320×240 RGB888,缩放采用 2×2 均值 `(sum+2)>>2`。该运算与 OpenCV 的 `INTER_AREA` 缩放**逐字节一致**,我们在板上用 DMA 往返对比验证了这一点(byte-exact)。核本身 LUT 1,124、FF 353、BRAM 2、DSP 0。

该核用于基准评测:对比 ARM 软件实现与 PL 实现的耗时(见 7.5 节)。在目前的系统结构中,摄像头输出的是 JPEG 压缩帧,ARM 并不解码,PC 端直接解码并推理,因此预处理核没有进入主控制环。这是我们对工程收益作出的取舍,我们没有为了“显得用了 FPGA”而强行接入。

5.4 资源、时序与功耗

表 5-3 最终设计资源占用(100 MHz,xc7z020clg400-1)

资源	用量	总量	占比
Slice LUT	4,428	53,200	8.32%
Slice 寄存器	6,740	106,400	6.33%
Slice	2,216	13,300	16.66%
Block RAM(RAMB36)	3	140	2.14%
DSP48E1	5	220	2.27%
用户 IO	9	125	7.20%
BUFG	1	32	3.13%

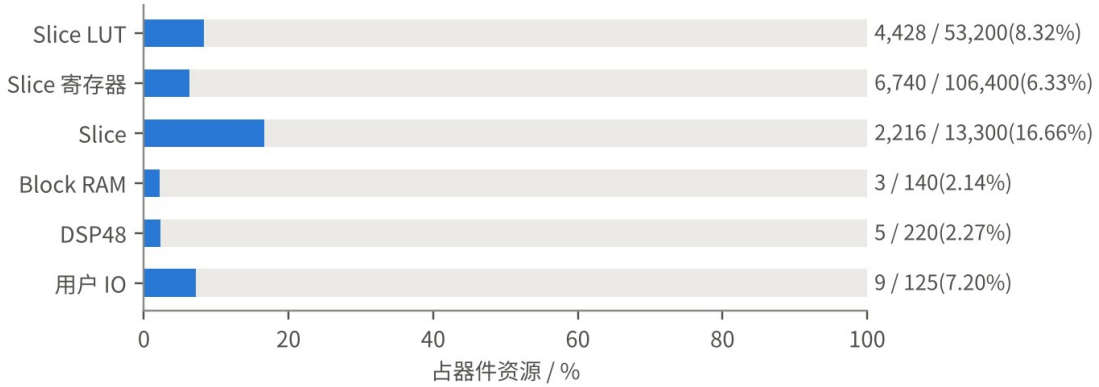


图 5-1 PL 资源占用

时序方面,工作频率 100 MHz,最差建立时间裕量 WNS=+1.024 ns,最差保持时间裕量 WHS=+0.031 ns,TNS 与 THS 均为 0,布线失败数为 0。据此估算最高可运行约 111 MHz。

功耗为 Vivado 在布线后的**估算值**(置信度为 Low,输入翻转率采用默认值),片上总功耗 1.432 W,其中动态 1.294 W、静态 0.138 W。功耗绝大部分来自 PS 端的 ARM 与 DDR 接口(1.256 W,占 88%);**PL 中全部用户逻辑合计约 0.038 W,占 2.7%**。这说明把确定性任务放进 FPGA 几乎不增加能耗。

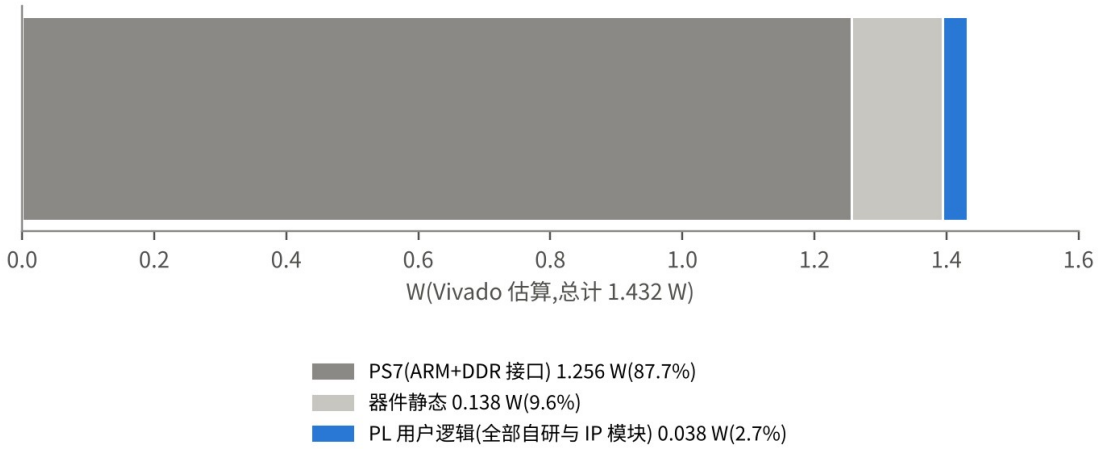


图 5-2 片上功耗构成(Vivado 估算)

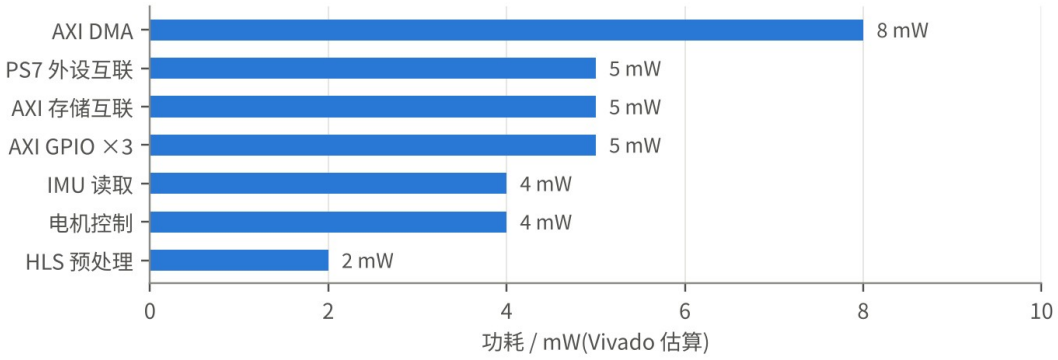


图 5-3 PL 各模块功耗(Vivado 估算)

6 软件与算法设计

6.1 视觉感知服务

PC 端运行 `server_v2.py`:接收帧头(帧号、长度)与 JPEG 数据,解码后用 YOLOv8n 推理,置信度阈值取 0.15。回传 28 字节:帧号、目标数、最危险目标的水平位置 cx、面积占比 area、推理耗时、底边位置 ybot 与类别。解码失败时回传“坏帧”标记,ARM 端跳过该帧,不执行任何动作。

目标选择策略。 画面中可能同时有多个物体。我们按“面积 ÷ 该类阈值”最大者选取最危险目标:小物体(水瓶、杯子、背包等)阈值为 0.06,大物体(人、椅子、长椅等)

阈值为 0.10。这样既不会让远处的大物体抢走小物体的位置,也避免了画面边缘一小块人影造成误触发。

6.2 触发判据的选择过程

起初我们尝试用目标框底边位置(ybot)估计距离。实测发现摄像头安装位置很低,画面下沿以下的地面不可见,ybot 在 0.5 米到 1.0 米的范围内都贴近 1.0,无法区分远近;而目标面积随距离的平方反比变化(0.5 米时约 0.10~0.15,1.0 米时约 0.03),规律明确。因此最终采用**面积占比**作为触发判据,ybot 仅保留在日志中供参考。这一过程体现了“先测量、再设计”的工程方法。

6.3 航向保持与闭环转向

直行航向保持。 起始时记录当前偏航角作为目标航向,之后每个控制周期按 `c = kp·(目标航向 − 当前偏航)` 对左右电机作差动修正(kp=15,修正量限幅 250),基础占空比 800。

脉冲式闭环转向。 TT 电机在小占空比下无法转动,比例控制会出现堵转嗡鸣。因此转向采用“满功率短脉冲 + 停车读角 + 再估计下一次脉冲时长”的方式:约 120 ms 的满功率脉冲对应约 20° 转角(约 6 ms/°),每次脉冲后读取 PL 偏航角,据剩余角度计算下一次脉冲时长,若进展不足则自动加大脉冲。

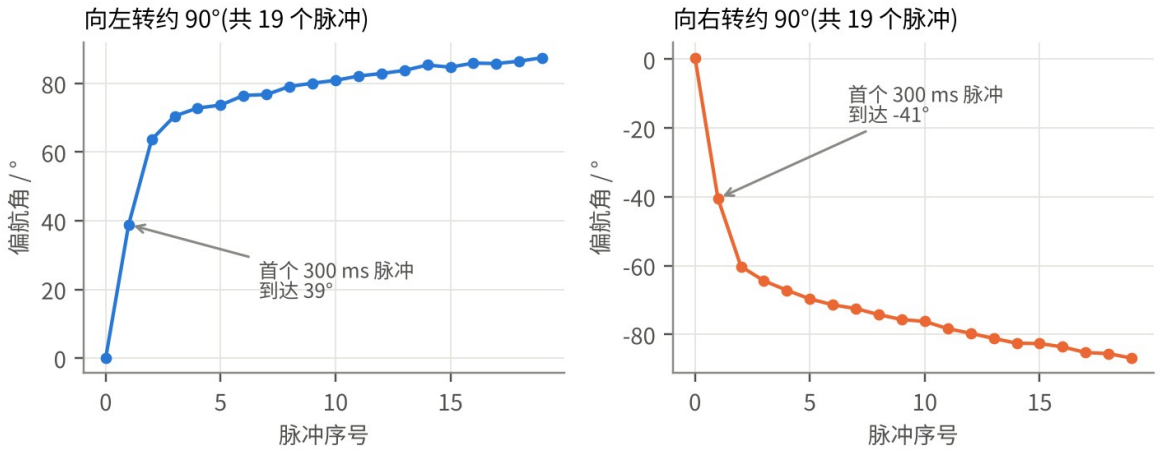


图 6-1 闭环转向收敛过程(左转与右转约 90°,数据来自 closed_loop_bursts.csv)

6.4 安全机制

- **PL 看门狗:** 心跳超过 200 ms 自动刹车(见 5.2 节)。
- **坏帧与超时处理:** JPEG 解码失败或网络超时时不执行动作,保持当前状态或停车。
- **连续转向保护:** 连续 4 次转向后仍判定有障碍,则停车退出,避免原地打转。
- **重复运行保护:** 在同一 Jupyter 内核中重复创建对象时,先停止旧的心跳线程、复用已加载的比特流、释放摄像头。早期版本因重复下载比特流而引起整板死机,这一保护解决了该问题。

7 实验与结果分析

7.1 实验设置

实验平台为第 4 章所述小车,运行 PYNQ 3.1.1,PL 时钟 100 MHz。PC 与 PYNQ 通过网线直连,IP 分别为 192.168.2.1 与 192.168.2.99。所有统计均保留原始 CSV 日志,并以 P50/P95/P99/max 报告,不仅给出平均值。时间戳取自 PL 硬件计数器。除特别说明外,延迟统计已剔除每次运行的前 3 帧(YOLO 预热)。

7.2 PL 时间戳确定性

我们记录了 1497 个相邻 IMU 样本的 PL 时间戳间隔(图 7-1):**全部为 2000 μs,标准差为 0**。这是纯硬件定时的直接体现。在软件定时的系统中,同样的统计通常会出现数百微秒级的抖动。

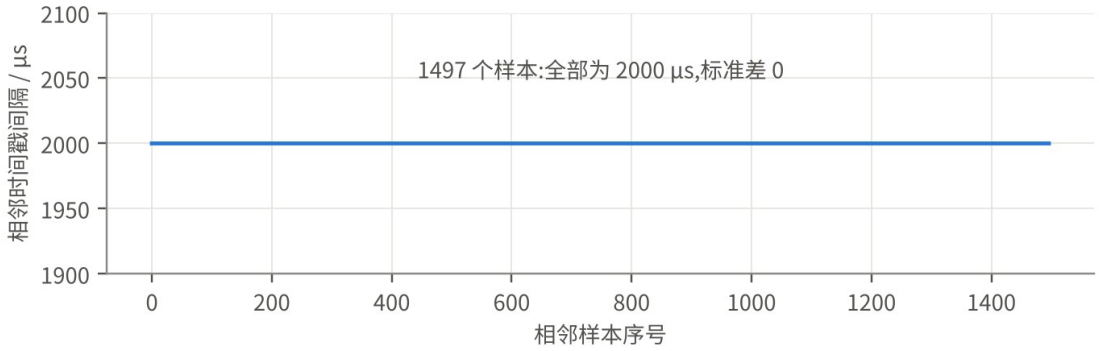


图 7-1 PL 时间戳相邻间隔(1497 个样本)

7.3 PS→PL 控制写延迟

PS 通过 AXI-Lite 向 PL 写一次寄存器(电机命令)的耗时,共 5000 次测量(图 7-2,表 7-1)。主体分布集中在 22~26 μs。偶发的长尾(共 11 次超过 100 μs,最大 702 μs)来自 Linux 的非实时调度,与 PL 无关。**这也正是我们把看门狗放进 PL 的原因:即使软件偶发抖动,硬件仍能保证失联刹车。**

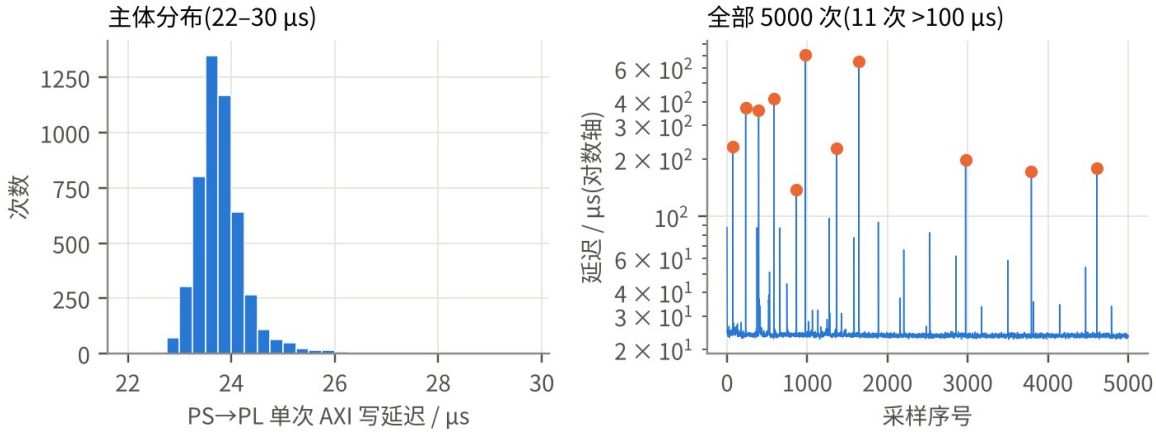


图 7-2 PS→PL AXI 写延迟(5000 次)

表 7-1 PS→PL 写延迟统计

样本数	最小	P50	P95	P99	P99.9	max
5000	22.58 µs	23.74 µs	24.86 µs	33.64 µs	233.2 µs	702.3 µs

7.4 闭环转向精度

在 10 次避障试验中共发生 23 次转向事件,转向最终误差见图 7-3。**中位数绝对误差 1.77°,21/23 次不超过 3°**;另有 2 次误差较大(15.4°、28.1°),均发生在车辆一启动就检测到很近的椅子的试验中,转向在时限内未收敛,我们如实保留了这两个点。独立的 8 次转向测试中,平均绝对误差 1.67°,最大 3.09°。

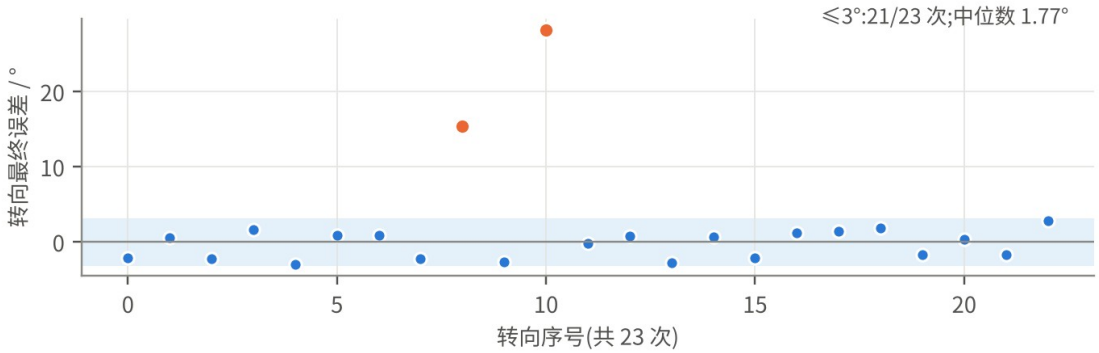


图 7-3 23 次转向的最终误差(蓝色 ≤3°,橙色 >3°)

7.5 PL 图像预处理加速

在同一张 640×480 图上对比 ARM 软件实现(OpenCV `INTER_AREA`,1000 次)与 PL 实现(核心 500 次),结果见图 7-4、7-5 与表 7-2。PL 核心 P50 为 3.235 ms,相对 ARM 的 13.013 ms 加速约 **4.0 倍**;计入 DMA 搬运的端到端 P50 为 6.91 ms,加速约 **1.88 倍**。并且 PL 实现的耗时分布更窄(P99 与 P50 仅差 0.15 ms)。

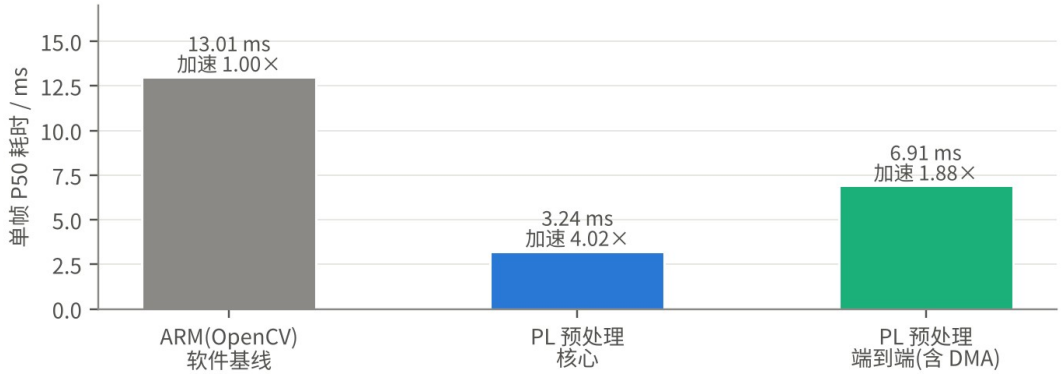


图 7-4 预处理单帧 P50 耗时对比

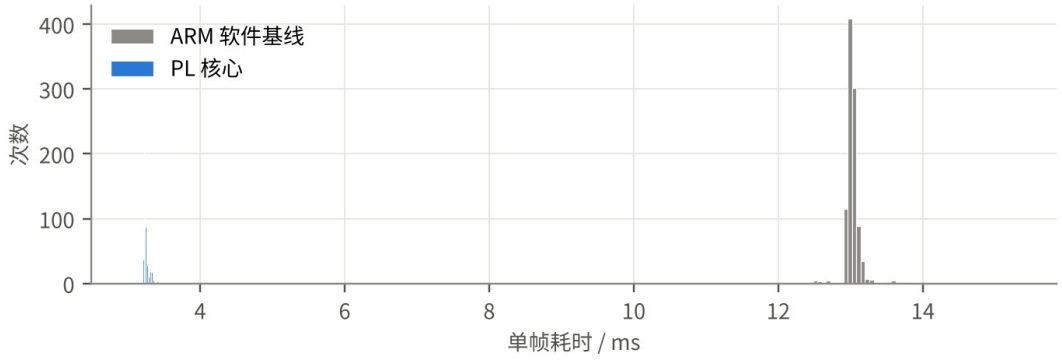


图 7-5 预处理耗时分布(ARM 基线与 PL 核心)

表 7-2 预处理耗时统计(ms)

实现	样本数	P50	P95	P99	max
ARM 软件基线	1000	13.013	13.163	13.712	15.146
PL 核心	500	3.235	3.324	3.381	3.547
PL 端到端(含 DMA)	—	6.91	【待补】	【待补】	【待补】

> **诚实说明:** 端到端加速只有 1.88 倍,是因为 DMA 搬运与缓冲拷贝占用了相当时间。而且在我们的结构里,摄像头给的是 JPEG,ARM 本来就不做缩放,因此预处理核没有进入主环。我们把它定位为“可复现的基准评测”,并如实报告,而不是夸大。

7.6 端到端避障延迟

端到端延迟定义为:ARM 取到摄像头画面(JPEG)→ 经网络发到 PC → YOLO 推理 → 结果返回 → ARM 写出电机指令。不含电机物理响应时间。合并 10 次试验共 253 个直行帧的统计见图 7-6、7-7 与表 7-3:**P50 14.13 ms、P95 18.93 ms、P99 23.48 ms、max 27.32 ms**。其中 PC 端推理占大头(P50 12.47 ms),其余约 1.7 ms 为 JPEG 传输、网络往返与 PL 写入。

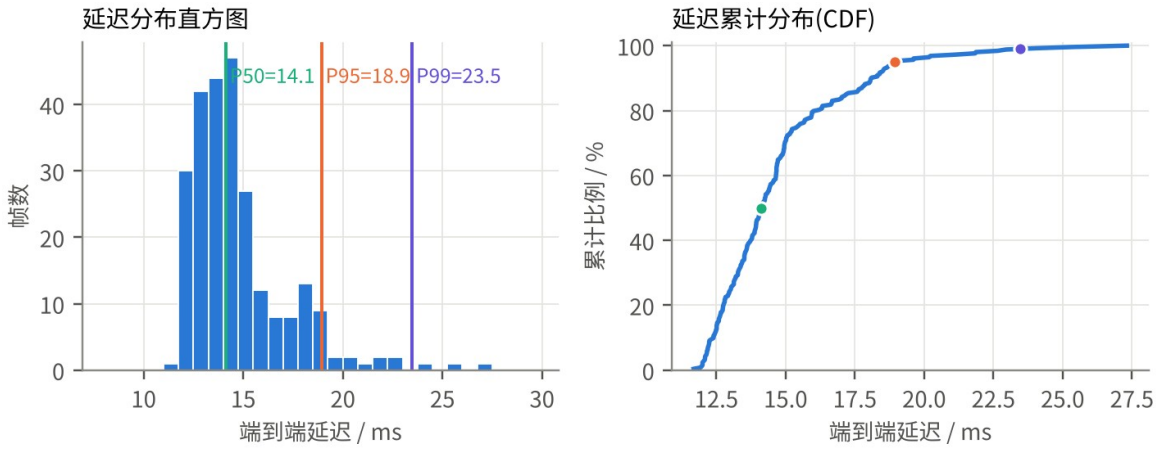


图 7-6 端到端延迟分布(直方图与累计分布)

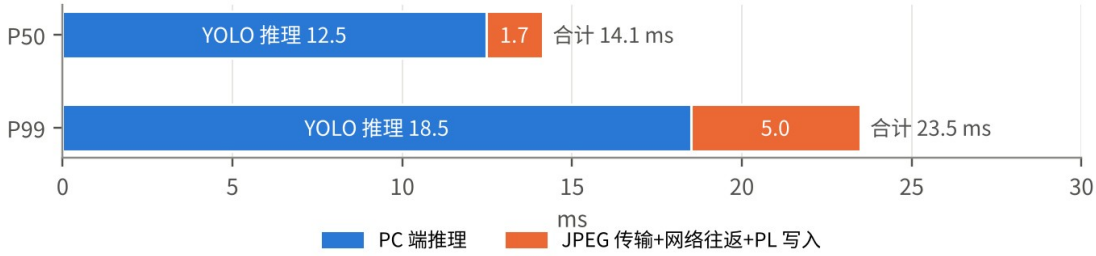


图 7-7 端到端延迟构成

表 7-3 端到端与推理延迟统计(ms;端到端 n=253 个直行帧,推理 n=271 个预热后帧)

指标	均值	P50	P95	P99	max
端到端延迟	14.74	14.13	18.93	23.48	27.32
PC 端推理	13.00	12.47	16.55	18.49	20.63

在 25 fps 的处理节奏下(帧间隔约 40 ms),端到端 P99 小于一个帧周期,说明系统没有因为延迟波动而丢失控制节拍。297 帧中仅有 1 帧因摄像头坏帧被丢弃(约 0.3%),链路超时 0 次。

7.7 避障功能验证

图 7-8 是一次完整的避障过程(试验 3):小车沿目标航向直行,偏航角在 ±5° 内波动;约 1.2 s 时目标开始进入视野,框面积逐步增大;面积超过水瓶阈值 0.06 时,小车判定障碍,向右转向避开。目标框面积在个别帧中为 0,是检测器偶发漏检造成的,我们没有在作图时人为平滑它。

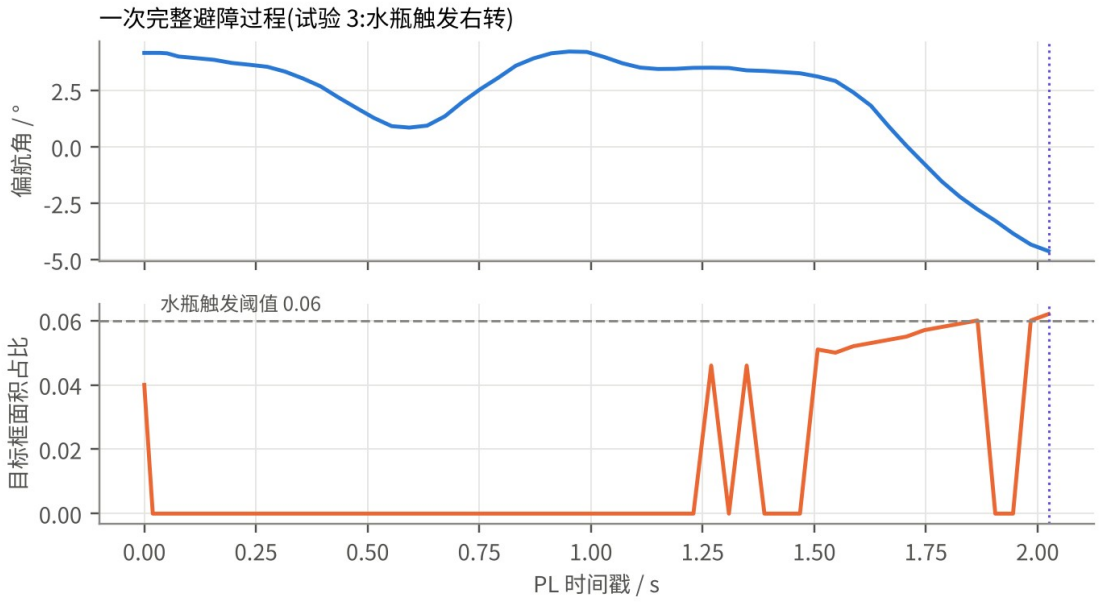


图 7-8 一次完整的避障过程(PL 时间戳为横轴)

在 10 次试验(共 297 帧)中,系统共触发 23 次转向,涉及人、椅子、盆栽、长椅、猫、汽车与水瓶(图 7-9)。需要说明:由于试验环境中桌边有盆栽与椅子,多数转向是由这些

背景物体触发的,这恰好说明系统对多类别目标都能响应,但**这些并不能算作水瓶避障成功**。由水瓶(类别 39)触发的转向共 2 次(试验 3、试验 10,触发面积分别为 0.062、0.060),转向误差分别为 -2.71° 与 -2.24°,均未发生碰撞。

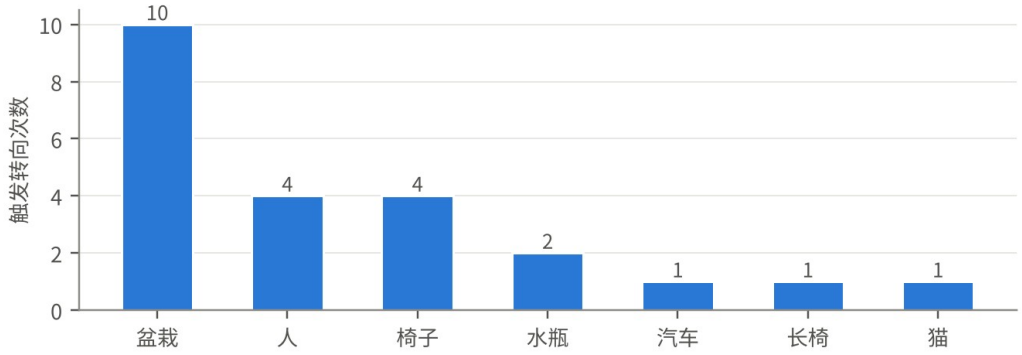


图 7-9 10 次试验中各类别触发转向次数

表 7-4 避障成功率统计

障碍物	试验次数	成功避开	擦碰	未触发	成功率
水瓶(正式统计)	【待补】	【待补】	【待补】	【待补】	【待补】
水瓶(功能验证,已有)	2 次触发	2	0	—	—

7.8 指标达成情况总结

表 7-5 主要实测指标汇总

指标	数值	数据来源
PL 时间戳间隔	2000 μs,σ=0(n=1497)	pl_imu_ts_step.csv
PS→PL 写延迟	P50 23.74 / P99 33.64 / max 702.3 μs	ps_pl_write_latency.csv
端到端延迟	P50 14.13 / P95 18.93 / P99 23.48 / max 27.32 ms	trial_bottle_*.csv
转向误差	中位数 1.77°,21/23 次 ≤3°	trial_bottle_*.csv
预处理加速	核心 4.0×,端到端 1.88×	pre_arm_baseline.csv 等
资源占用	LUT 8.32% / FF 6.33% / BRAM 2.14% / DSP 2.27%	Vivado 利用率报告
时序	WNS +1.024 ns,WHS +0.031 ns	Vivado 时序报告
功耗(估算)	1.432 W(PL 用户逻辑约 0.038 W)	Vivado 功耗报告

7.9 优化过程与前后对比

本节把设计过程中做过判断、且有数据支撑的优化放在一起,给出优化前后的性能与资源对比。没有“优化前”数据的改动(例如电机死区补偿、航向保持增益)不放进表中,避免编造对比。

表 7-6 优化前后对比

项目	优化前	优化后	收益 / 代价
图像预处理 (640×480→320×240)核心耗时 P50	ARM 软件 (OpenCV)13.013 ms	PL 核(HLS+DMA)3.235 ms	加速 4.0×;P99 13.71→3.38 ms;LUT +1,124(2.1%)、FF +353、BRAM +2、DSP 0
图像预处理 端到端 P50	13.013 ms	6.91 ms(含 PS 侧准备)	1.88×;P95/P99/max 待补
触发判据	框底边位置 ybot:近距离(0.5~1 m)几乎恒为 0.96~1.0,无法区分远近	框面积占比,按类别设阈值(小物体 0.06、大物体 0.10)	水瓶约 0.65~0.7 m 触发;背景大物体不再被小物体阈值误判
重复运行稳定性	同一内核重复加载 Overlay:100% 掉线	同一比特流只加载一次、先关旧线程	重复运行不再掉线(未做长时统计)

预处理核的收益需要放在系统里看:它把一帧的预处理从 13 ms 降到 3.2 ms,但当前主控制环里摄像头输出 JPEG,由 PC 直接解码,这个核并没有进入端到端路径,所以端到端的 14.13 ms 里没有它的贡献。我们把它如实归为“已验证的加速能力”,而不是“已落地的系统收益”。

8 创新点与技术亮点

1. **硬件时间戳:** 每个 IMU 样本由 PL 以微秒时间戳标记,间隔标准差为 0,为后续视觉—惯性数据对齐提供了可信的时间基准。
2. **PL 硬件看门狗:** 失联自动刹车逻辑独立于操作系统,实测可在 200 ms 内触发并点亮指示灯,是系统安全性的硬件兜底。
3. **PL 闭环偏航反馈:** 转向使用 PL 偏航角做闭环,23 次转向中位数误差 1.77°。
4. **以测量驱动设计:** 触发判据由 ybot 改为面积,是在测得 ybot 饱和后作出的,避免了凭想象设计。
5. **极低资源与功耗:** 全部 PL 逻辑占 8.32% LUT、约 38 mW,具备继续扩展的空间。
6. **诚实、可复现的评测:** 全部指标给出 P50/P95/P99/max 与原始日志,边界与局限在报告中明示。

9 工程问题与经验

9.1 开发板反复掉线

调试中开发板多次无响应,必须断电才能恢复。起初我们怀疑供电,更换电源、给摄像头单独供电均未根治。后来通过对照发现:**每次在同一内核里重复运行包含 Overlay 加载语句的单元,100% 掉线。** 根因是重复下载比特流时,旧的心跳线程仍在每 20 ms 读

写 AXI 寄存器,PL 重配置期间总线事务无法完成,PS 被卡死。修复方法是:同一比特流只加载一次;重复创建对象时先停旧线程;并释放摄像头。修复后重复运行不再掉线。

9.2 IMU 读取恒为 0

IMU 初始读数全为 0,即使拔掉传感器也没有 NACK,引脚电平恒为 0。根因见 4.2 节块设计综合为独立单元,三态信号没有映射到引脚。在 RTL 中显式例化 `IOBUF` 后恢复正常。该问题在仿真中无法发现,强调了上板验证的重要性。

9.3 低占空比下电机堵转

TT 电机加塑料轮,低占空比下只嗡鸣不转。我们放弃了比例转速控制,改为“满功率短脉冲 + 读角”的脉冲式闭环,并加了死区补偿,转向误差降到 1~3°。

9.4 视觉触发判据

见 6.2 节:ybot 因摄像头过低而饱和,改用面积;并按类别区分阈值,避免背景物体误触发。

10 大模型协作、技能包与复现

10.1 协作方式与分工

本作品的开发全程与大模型协作。分工是:指导教师带领学生搭建小车并在硬件上执行命令,把输出原样回传;大模型负责写 RTL / HLS / Python 代码和测试平台、读日志与数据并提出可检验的假设、设计实验步骤、汇总数据并撰写报告与教程。大模型无法直接访问板子,所有“板上结论”都来自回传的真实输出;关乎取舍的决定(是否补测、测试对象、场地限制)由人做出,模型只给出代价与建议。

完整的交互记录见提交包 `report/llm_log/`。需要说明的是:长会话经过上下文压缩,压缩前的逐字对话不在可读文件中,因此记录里“用户原话”为逐字,“回答要点”和“纠错”为按压缩摘要整理的转述,并已逐条注明;完整原始导出将随提交包补充。

10.2 自我纠错轨迹(节选)

表 10-1 大模型协作中的自我纠错

模型最初的判断或产物	发现的证据	修正
开发板掉线,先怀疑供电与摄像头	换电源、摄像头单独供电均未根治;用户对照发现“重启内核就不掉线”	改为软件根因(重复加载 Overlay),代码修复
用 ybot(框底边)判断距离	近距离 ybot 恒≈1.0,失去分辨力	改用框面积占比并按类别设阈值

模型最初的判断或产物	发现的证据	修正
把 10 次试验当作“水瓶成功”	逐次检查触发类别,23 次转向中只有 2 次由水瓶触发	加入 expect_cls 告警;水瓶成功率保留“待补”;更正此前“1 次”的说法
报告中把序号锁描述为“前后各加一次、偶数有效”	对照 imu_reader.v:实际为同周期一次性提交并加 1	更正报告与技能包说明,以代码为准
报告推理耗时与复算脚本不一致 (12.47 vs 12.40 ms)	追查口径:报告用预热后全部帧,脚本最初只用直行帧	统一口径并写明

10.3 技能包与提炼过程

技能包位于提交包 `skill/`,每项都写明适用场景、使用方法、已验证效果、失效条件和提炼来源。其中四个工具与题目无关,面向 PYNQ / Zynq 通用场景,并带有可运行的离板自测:

表 10-2 技能包条目

条目	提炼自	验证
pynq_safe_overlay:同一比特流只加载一次、先关旧线程	开发板反复掉线	实板重复运行不再掉线;离板自测通过
latency_report:原始耗时 → P50/P95/P99/P99.9/max	比赛要求尾延迟;PS→PL 写 P99.9 达 233 µs	复算与报告一致
golden_compare:与软件黄金参考逐字节比对	HLS 核需证明与 INTER_AREA 一致	对 OpenCV 交叉验证;注入 1 LSB 错误可检出
seqlock_snapshot:同周期提交 + 读-数-读快照	时间戳与偏航角需同一样本	朴素读 299/300 次撕裂,seqlock 0 次
prompts/:4 个提示词工作流;pitfalls.md:8 条踩坑	全部开发过程	逐条注明是否在实板验证

边界:离板自测只证明逻辑正确;实板结论来自一块 PYNQ-Z2 的使用反馈,没有大样本统计;这些工具尚未在第二块板卡或第二个题目上复用过,“通用”是按其不依赖本题目代码来判断的。

10.4 器件、工具与开源

- **器件**:AMD Zynq-7000 XC7Z020-1CLG400C(PYNQ-Z2),PYNQ 镜像 3.1.1。
- **工具版本**:Vivado 2024.1、Vitis HLS 2024.1。选题指南推荐 2026.1 或 2025.2,本作品没有用推荐版本构建,换版本需重新生成 IP 并重跑构建脚本,我们没有验证过。
- **开源**:提交包采用 MIT 协议,目录结构按选题指南推荐组织(src / sim / build / board / data / skill / report),全部目录与文件名为纯英文。

- **复现**:不依赖 Vivado 和板子的部分(HLS C 仿真、IMU 镜像仿真、全部数据复算、技能包自测)一条命令即可复跑:`bash build/check_repo.sh`;上板步骤见 `board/README.md` 与配套《使用教程》。比特流的文本化重建脚本已提供,但尚未在作者环境中执行过(见 `build/README.md`)。

11 局限性与改进计划

表 11-1 当前局限与改进方向

局限	影响	改进方向
检测依赖 COCO 预训练类别	未覆盖的物体(纸箱、墙板等)不会被识别	针对比赛场景微调模型;或增加超声波/深度传感器兜底
摄像头安装过低	地面不可见,无法用底边估计距离	抬高并略微下倾摄像头;改用面积+多帧融合
预处理核未接入主环	FPGA 对视觉通路的加速未体现在端到端	改为 PL 摄像头直采(DVP/MIPI)后接入预处理与更多算子
视觉与惯性仅做时间对齐	尚未做状态估计级的融合	在 PS 或 PL 中加入互补滤波/扩展卡尔曼滤波
功耗为估算值	非实测	使用功率计测量整机功耗
检测在 PC 上完成	依赖网络	迁移至 AMD Ryzen AI PC 或板载加速
避障成功率统计样本偏少	统计置信度有限	按附录 D 补充每类 10 次以上试验

12 总结与展望

本作品围绕“让 FPGA 做确定性的事”这一思路,在 PYNQ-Z2 上实现了一台视觉—惯性协同的自主避障小车。PL 中自研的 IMU 读取核与电机控制核提供了零抖动的硬件时间戳、微秒级的控制通路以及不依赖软件的失联刹车;PC 端的 YOLO 检测给出语义信息;ARM 端以 Python 完成灵活的避障策略。实测端到端延迟 P99 为 23.48 ms,转向误差中位数 1.77°,PL 资源占用仅 8.32% LUT。

我们同样清楚作品的边界:视觉通路尚未真正由 FPGA 加速到端到端,检测类别受预训练模型限制,避障成功率的正式统计仍需补充。这些在报告中均已如实写明。后续工作将聚焦于:摄像头直采与 PL 预处理接入主环、视觉—惯性的状态估计级融合、模型迁移至 AMD Ryzen AI PC,以及更大样本的统计验证。

参考文献

[1] Redmon J, Divvala S, Girshick R, Farhadi A. You Only Look Once: Unified, Real-Time Object Detection. CVPR, 2016.

[2] Jocher G, Chaurasia A, Qiu J. Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics, 2023.

[3] Xilinx. Zynq-7000 SoC Technical Reference Manual (UG585).

[4] Xilinx. AXI DMA LogiCORE IP Product Guide (PG021).

[5] AMD. Vitis High-Level Synthesis User Guide (UG1399).

[6] InvenSense. MPU-6000 and MPU-6050 Register Map and Descriptions.

[7] PYNQ Project. PYNQ Documentation. https://pynq.readthedocs.io

附录 A 复现步骤摘要

完整的、面向初学者的操作步骤见配套文档《使用教程》。此处仅给出主要流程:

1. PC 与 PYNQ 用网线直连,PC 设置 192.168.2.1,PYNQ 为 192.168.2.99。
2. PC 上运行 `python server_v2.py`,等待“等待连接...”。
3. 浏览器打开 `http://192.168.2.99:9090`,上传 `car_pl.py`、`pipeline.bit`、`pipeline.hwh`。
4. 在 Jupyter 中运行:

```
from car_pl import *
car = Car("pipeline.bit")
vis = Vision("192.168.2.1")
run_avoid(car, vis, seconds=7)
```

附录 B PC 与 PYNQ 通信协议

方向	格式	字段
PYNQ → PC	!II + JPEG 字节	帧号(uint32)、JPEG 长度(uint32)、JPEG 数据
PC → PYNQ	!IIfffff(28 字节)	帧号、目标数、cx、area、infer_ms、ybot、cls
坏帧	目标数 = 0xFFFFFFFF	ARM 跳过该帧

附录 C 原始数据文件清单

文件	内容
pl_imu_ts_step.csv	PL 时间戳相邻间隔(μs)
ps_pl_write_latency.csv	PS→PL 写延迟(μs,5000 次)

文件	内容
pre_arm_baseline.csv	ARM 预处理耗时(ms,1000 次)
pl_preproc_core.csv	PL 预处理核心耗时(ms,500 次)
closed_loop_bursts.csv	闭环转向脉冲过程
trials.csv / trial_bottle_*.csv	避障试验汇总与逐帧日志

附录 D 待补数据清单

下列内容在场地与设备允许后补测,补测后只需替换对应表格与数字:

- 水瓶避障正式统计(建议 ≥10 次):表 7-4。
- PL 预处理端到端 P95 / P99 / max:表 7-2。
- 网络吞吐与丢包实测(目前吞吐由帧率与帧大小推算,约 3.4 Mbit/s)。
- 参赛队信息(队名、学校、队员、指导教师):封面。
- 可选:整机功耗实测;AMD Ryzen AI PC 上的推理延迟对比。
- 提交包中的待补项:Verilog 测试平台在 Vivado xsim 的运行日志、pipeline.bit / pipeline.hwh 、 Vivado 时序报告原文件、文本化块设计(运行 build/export_from_vivado.tcl)、大模型协作的完整原始对话导出、LICENSE 中的队名。